

객체지향언어1 [분반7반]

[11주차 과제3:예제7-6번,9-4번문제풀기]

교수명:황기태

2371214 이지형

트랙:빅데이터트랙&모바일소프트웨어 트랙

1. 예제7-6 실습문제(소요시간:30분)

-Light.h(헤더파일)-

```
#ifndef LIGHT_H //LIGHT_H가 정의되어 있지 않다면 아래 내용을 포함

#define LIGHT_H //LIGHT_H를 정의하여 이 파일이 중복으로 읽히는 것을 방지

class Light { //빛을 추상화하여 관리하고 제어하기위한 클래스 Light 선언

    int light; //객체가 가지고 있는 빛의 세기를 표현하는 정수형 변수

public: //외부에 공개하기위한 public접근지정자 지정

    Light(int light); //입력된 빛의 값으로 세기를 설정하여 객체를 생성하는 생성자

    Light(); //별도의 입력이 없을 경우 기본 밝기 1을 가진 객체를 생성하는 기본생성자

    Light& operator+=(Light& p);

    //상대객체의 빛의 세기를 흡수하고, 자신의 상태를 반환하는 연산자 중복

    Light& operator+=(int light);

    //빛의 세기를 정수 값으로 직접받아 현재세기에 합산하는 연산자 중복

    void operator~(); //현재 객체가 가진 빛의세기를 출력하는 연산자~ 연산자중복

    Light operator--(int x); //세기를 1낮추지만 낮추기전의 세기를 반환하는 후위연산자 중복

    friend bool operator ==(Light p, int x);

    //빛의 세기가 일치하는지 friend형태로 외부에서 판별하는 연산자중복

};

#endif // LIGHT_H 정의의 끝
```

-Light.cpp(클래스 구현부)-

```
#include <iostream> //표준 입출력기능을 활용하기위한 라이브러리

using namespace std; //std 이름공간에 선언된 모든 이름에 std생략

#include "Light.h" // 작성한 Light 클래스의 선언부인 헤더파일을 불러옴


Light::Light(int light) { //매개변수를 가지는 생성자 구현부

    this->light = light; //입력된 값을 해당객체의 빛의세기로 할당

}

Light::Light() { //클래스의 기본생성자 구현부

    light = 1; //기본 에너지 상태를 1로 설정

}

//객체참조리턴을 통해 합산된 자기자신을 리턴하는 +=연산자 중복 구현부

Light& Light::operator+=(Light& p) {

    light += p.light; //두 빛의 세기를 합산하여 자신의 값으로 저장

    p.light = 0; //합산당한 객체의 빛의세기를 0으로 설정함

    return *this; //수정된 객체자신을 반환하여 연쇄적인 연산을 가능하게 함

}

//정수형태의 빛의 형태를 자신에게 합산하는 +=연산자 중복 구현부

Light& Light::operator+=(int light) {

    this->light += light; //두 빛의 세기를 합산하여 자신의 값으로 저장

    return *this; //수정된 객체자신을 반환하여 연쇄적인 연산을 가능하게 함

}
```

//a.operator~()형태로 컴파일러가 변환,빛의크기를 출력하는 ~ 연산자중복

```
void Light::operator ~() {
```

```
    cout << "빛 크기 " << light << endl; //현재 빛의 세기를 텍스트로 출력함
```

```
}
```

//후위연산자로서 전위연산자와 비교하기위해 매개변수값을 포함

```
Light Light::operator--(int x) {
```

```
    Light temp; //세기가 줄어들기전의 밝기를 임시로 보관할 임시객체 생성
```

```
    temp = *this; //임시객체의 기존 자신의 값을 저장함
```

```
    --light; //빛의세기를 1단계 줄임
```

```
    return temp; //참조리턴을 할시 소멸되는 객체를 리턴하기에 객체리턴 실시.
```

```
}
```

//operator==(p,x)형태로 컴파일러가 변환,객체외부에 friend형태로 선언

```
bool operator ==(Light p, int x) {
```

```
    if (p.light == x) //객체p의 빛의 세기와 x가 같은지 판단
```

```
        return true; //수치가 일치한다면 true값을 리턴
```

```
    else //객체p의 빛의 세기와 x가 같지 않은 경우
```

```
        return false; //수치가 일치하지 않으므로 false값을 리턴
```

```
}
```

-main.cpp(메인함수)-

```
#include <iostream> //표준 입출력기능을 활용하기위한 라이브러리
```

```
using namespace std; //std 이름공간에 선언된 모든 이름에 std생략
```

```
#include "Light.h" // 작성한 Light 클래스의 선언부인 헤더파일을 불러옴
```

```
int main() {
```

```
    Light a(10), b(20), c, d(0); //객체 c는 기본생성자 생성으로 빛의세기가1, d의 크기는 0이됨.
```

```
    (a += b) += 10; //+=연산자중복을 통해 빛 a에 b를 더하면 b의 크기를 잃고 0이 됨.
```

```
    ~a, ~b; ~c; //~연산자 중복을 통한 빛 a,b,c의 크기 출력
```

```
    d = c--; // --후위연산자를 통해 d의 빛 크기는 1이 되고 c의 빛크기 1감소, 0이 됨.
```

```
    ~d; //~연산자 중복을 통한 빛 d의 크기를 출력
```

```
    if (c == 0) //c의 빛의 세기가 0과 같은지 비교하여 조건문을 작동
```

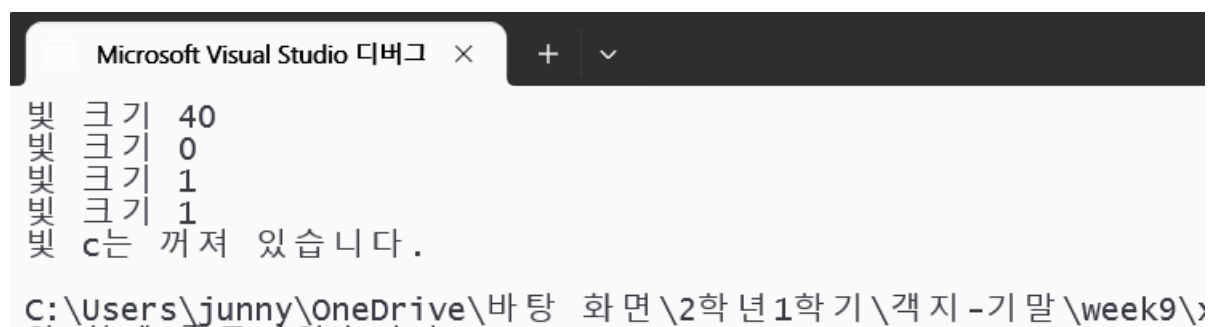
```
        cout << "빛 c는 꺼져 있습니다." << endl; //c가 0이라면 조건문 출력
```

```
    if (d == 0) //d의 빛의 세기가 0과 같은지 비교하여 조건문을 작동
```

```
        cout << "빛 d는 꺼져 있습니다." << endl; //d가 0이라면 조건문 출력
```

```
}
```

-실행결과-



```
Microsoft Visual Studio 디버그 × + v
빛 크기 40
빛 크기 0
빛 크기 1
빛 크기 1
빛 c는 꺼져 있습니다.
C:\Users\junny\OneDrive\바탕 화면\2학년1학기\객지-기말\week9\
```

1. 예제9-4 실습문제(소요시간:1시간)

-AbstractPoint.h(부모클래스 헤더파일)-

```
#ifndef ABSTRACTPOINT_H //ABSTRACTPOINT_H가 정의되어있지않다면 헤더를 포함
#define ABSTRACTPOINT_H //ABSTRACTPOINT_H 헤더파일 정의

#include <string> //name변수타입을 사용하기 위한 string 헤더선언

class AbstractPoint { //각 회원객체가 가지는 이름과 포인트,적립률을 정의하는 클래스
protected: //상속관계의 하위클래스에서만 접근이 가능하도록 허용하는 접근 지정자

    std::string name; //포인트를 적립받는 회원의 이름을 저장할 string형식의 변수

    int point = 0; //누적포인트를 저장할 point변수를 만들고 초기포인트를 0으로 설정

    int ratio = 0; //결재금액에서 기본 적용될 적립비율 변수(%) 설정

public: //모두가 접근할수 있도록 하는 public 접근 지정자

    //회원정보및 기본 적립비율을 초기화하는 매개변수 생성자

    AbstractPoint(std::string name, int ratio);

    virtual void earn(int pay) = 0; //순수가상함수. pay금액에 대한 포인트를 적립

    virtual void show() = 0; //순수가상함수. 이름과 포인트 출력하는 함수

    int getPoint() { return point; }

    //외부에서 현재의 포인트를 리턴하여 전달하기 위한 접근 함수

};

#endif //헤더파일 구역의 끝
```

-AbstractPoint.cpp(클래스 구현부 파일)-

```
#include <iostream> //표준 입출력기능을 활용하기위한 라이브러리
#include <string> //회원의 이름을 string형태로 처리하기 위한 헤더
using namespace std; //std 이름공간에 선언된 모든 이름에 std생략

#include "AbstractPoint.h" //Abstractpoint.h 헤더파일 호출

//회원의 이름과 기본적립률을 저장하는 부모클래스의 생성자 구현부
AbstractPoint::AbstractPoint(string name, int ratio) {

    this->name = name; //매개변수로 들어온 이름을 클래스변수로 저장

    //기준금액 이하일때 적용할 일반적립율을 클래스변수에 저장

    this->ratio = ratio;
} //생성자 구현 종료
```

-Platinum.h(자식클래스 헤더파일)-

```

#ifndef PLATINUM_H //PLATINUM_H가 정의되어있지않다면 헤더를 포함

#define PLATINUM_H //PLATINUM_H 헤더파일 정의

#include "AbstractPoint.h" //부모클래스 성질을 상속받기 위해 부모헤더파일 포함


//AbstractPoint클래스를 상속받아 고유속성을 확장한 자식클래스

class Platinum : public AbstractPoint {

protected: //상속관계의 하위클래스에서만 접근이 가능하도록 허용하는 접근 지정자

    //적립율을 결정하기 위한 결재금액과 우대적립률을 나타내는 변수 초기화

    int baseMoney = 0, highRatio = 0;

public: //모두가 접근할수 있도록 하는 public 접근 지정자


//부모클래스의 생성자를 명시적으로 호출,기준금액을 설정하는 자식클래스 생성자선언

    Platinum(string name, int ratio, int baseMoney, int highRatio);

    //추상클래스의 순수가상함수 구현,pay금액에 대한 포인트 적립(오버라이딩)

    virtual void earn(int pay);

    //추상클래스의 순수가상함수 구현,현재까지의 적립금액을 출력(오버라이딩)

    virtual void show();

    //두객체의 포인트를 한쪽으로 완전히 병합하고,

    //병합당한 객체의 포인트를 초기화 하는 함수

    void combine(Platinum& p);

};

#endif //PLATINUM_H조건부 헤더파일 구역의 끝

```

-Platinum.cpp(클래스 구현부 파일)-


```

#include <iostream> //표준 입출력기능을 활용하기위한 라이브러리

#include <string> //회원의 이름을 string형태로 처리하기 위한 헤더

using namespace std; //std 이름공간에 선언된 모든 이름에 std생략


#include "Platinum.h" //Platinum.h 헤더파일 호출


//부모클래스의 생성자를 명시적으로 호출함으로서 기본생성자의 묵시적삽입을 방지
//+자식클래스 생성자구현

Platinum::Platinum(string name, int ratio, int baseMoney, int highRatio)

: AbstractPoint(name, ratio) {

    this->baseMoney = baseMoney; //적립률을 결정할수 있는 지출액 저장

    this->highRatio = highRatio; //우대적립률을 클래스 변수에 저장

}


void Platinum::earn(int pay) { //적립률을 결정,포인트를 저장해주는 earn함수 구현부

    if (pay > baseMoney) //지출액이 우대기준선을 넘을시, 우대적립률로 적립

        //우대비율을 곱해 포인트를 계산하고 포인트에 누적

        point += (pay * highRatio) / 100;

    else //지출액이 우대기준선을 넘지않을시, 일반적립률로 적립

        //일반비율을 곱해 포인트를 계산하고 포인트에 누적

        point += (pay * ratio) / 100;

}

```

```

void Platinum::show() { //회원의 이름과 포인트 수치를 출력하는 함수 구현부

    //부모의 멤버 변수(point)에 직접 접근하여 값을 가져오는 대신,

    // 캡슐화 원칙을 지키기 위해 안전하게 데이터를 반환하는 getPoint() 함수를

    //호출하여 활용함

    cout << name << "님*** " << "적립 포인트는 " << getPoint() << "점 입니다." <<
endl;

}

//매개변수로 전달받은 회원의 포인트를 내 포인트로 병합하고, 상대방의 포인트는
소멸시키는 함수구현

void Platinum::combine(Platinum& p) {

    point += p.point; // 참조를 통해 원본객체의 포인트를 현재 나의 수치에 합산

    p.point = 0; //참조를 통해 상대방 원본객체의 포인트를 복사본이 아닌 '실제 원
본'에서 0으로 초기화함

}

```

-main.cpp(메인함수)-

```

#include <iostream> //표준 입출력기능을 활용하기위한 라이브러리

#include <string> //회원의 이름을 string형태로 처리하기 위한 헤더

using namespace std; //std 이름공간에 선언된 모든 이름에 std생략

#include "Platinum.h" //Platinum.h 헤더파일 호출

int main() { //메인함수

    //생성자 의미:100000원 까지는 5%, 그보다 큰경우 7%

```

```

Platinum hwang("황기태", 5, 100000, 7), park("박채원", 5, 100000, 7);

hwang.earn(50000); //50000원 지출경우, 2500원 포인트 적립
hwang.earn(200000); //200000원 지출경우, 14000원 포인트 적립
park.earn(500000); //500000원 지출경우, 35000원 포인트 적립

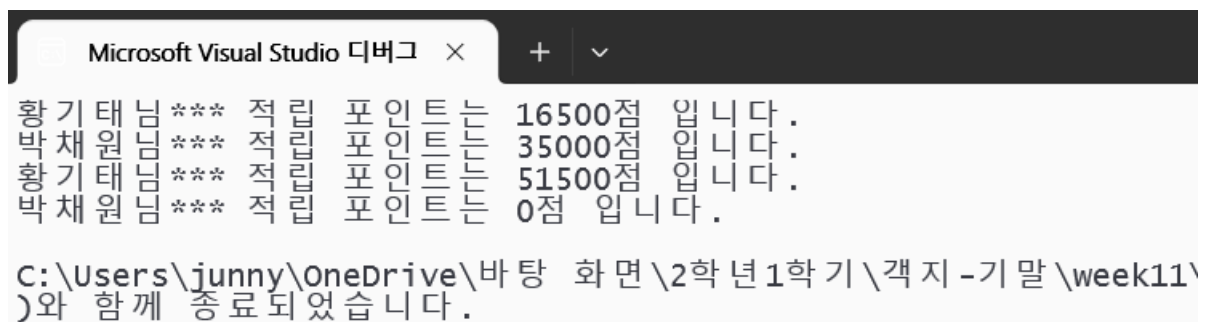
hwang.show(); //hwang객체의 현재 누적 포인트 출력
park.show(); //show객체의 현재 누적 포인트 출력

hwang.combine(park); //hwang회원에게 park의 포인트 결합
hwang.show(); //hwang객체의 현재 누적 포인트 출력
park.show(); //show객체의 현재 누적 포인트 출력

} //메인함수 종료

```

-실행결과-



The screenshot shows the output of a C++ program in the Visual Studio debugger. The output text is as follows:

```

황기태님 *** 적립 포인트는 16500점 입니다.
박채원님 *** 적립 포인트는 35000점 입니다.
황기태님 *** 적립 포인트는 51500점 입니다.
박채원님 *** 적립 포인트는 0점 입니다.

C:\Users\junny\OneDrive\바탕 화면\2학년1학기\객지-기말\week11\
)와 함께 종료되었습니다.

```

[과제3 수행 소감문-과제후기]

7-6번문제를 풀고 주석을 다는 것까지 총 1시간10분 정도의 시간이 소요되었습니다. 7장은 프렌드 함수와 연산자중복을 다루는 파트인데, +=연산자를 중복 정의하면서 Light& 참조 리턴 타입으로 사용해야 하는 이유와 각각의 연산자중복 함수를 작성할 때 리턴 타입을 어떻게 설정해야 하는지에 대해 확실하게 배울 수 있었습니다. 수정된 객체 자신(*this)을 참조형태로 반환해야만

`a += b += c;`와 같은 C++의 연쇄적인 연속연산이 자연스럽게 성립된다는 점을 이해했습니다. 다음에 연산자 중복과 연쇄계산이 들어간 문제를 풀이할시 자신감을 가지고 풀수있겠다고 생각하였습니다. 또한 -후위연산자를 구현할 때는 반대로 왜 객체참조리턴을 하면 안되는지도 이해하였습니다. 임시객체등을 함수내에서 만들었을 때는 꼭 해당 임시객체가 소멸된다는 것을 인지하고, 리턴타입을 정하는 습관을 가지게 된 것 같습니다. 또한 문제를 풀면서 `~a`는 `a.operator~()`형태로 변환되는데, `a~`는 어떻게 표현해야 될지에 대한 의문이 들어서 다시한번 교재를 공부한 결과, 표준문법상으로 `~`연산자는 원래 '전위 연산자'로만 쓸수 있다는 사실을 다시 한번 배웠습니다. -후위연산자 중복함수를 작성하면서는 후위연산자로서 전위연산자와 비교하기위해 매개변수값을 포함해야한다는 점 또한 다시한번 복습할 수 있는 기회가 된것같아 매우 유익하였습니다.

9-4번 문제는 8장 상속파트를 기반으로 하는 9장 가상함수와 함수 오버라이딩에 대해 다루는 파트입니다. 해당내용을 다시 한번 복습하고 문제 풀이 및 주석을 달기까지 총1시간30분정도의 시간이 소요되었습니다. 프로그램 실행과정을 살펴보자면, `main()`함수가 시작되면 `hwang("황기태", 5, 100000, 7)`과 `park("박채원", 5, 100000, 7)` 객체가 각각 메모리에 생성되는데, 이때 자식 클래스인 `Platinum`생성자가 생성되기 직전 명시적으로 호출해놓은 `AbstractPoint` 생성자가 먼저 호출되어 기본 변수인 `name, ratio`가 할당되고, `Platinum`생성자에서 `baseMoney, highRatio`가 최종적으로 할당됩니다. 해당과제의 설계에서 가장 주목 할부분은 '동적 바인딩'으로, 부모 클래스인 `AbstractPoint`에서 `earn()`과 `show()`함수를 `virtual`키워드가 붙은 순수 가상 함수(=0)로 선언한것이 출발점입니다. 일반적인 멤버 함수는 컴파일 단계에서 호출될 함수의 주소가 결정되는 '정적 바인딩'을 따르지만, 가상 함수는 프로그램이 실행되는 시점(Runtime)에 실제 객체의 타입을 확인하고 주소를 결정하는 '동적 바인딩'을 유도 합니다. 과제에서는 직접 부모 포인터를 선언하여 자신객체를 가르키는 업캐스팅을 사용하지는 않았지만, `Platinum`객체를 통해 함수를 실행할 때, 실행시점에 동적바인딩을 수행하여 실제 구현된 `Platinum` 클래스에 오버라이딩된 구체적인 함수를 찾아가 실행되는 과정을 거칩니다.

또한, `combine`함수에서는 함수의 매개변수를 참조를 통해 받아오기 때문에(`Platinum& p`) 복사본이 아닌 실제 원본객체의 포인터가 0으로 초기화되도록 구현 하였습니다.

이번 9장4번 과제를 수행하면서 느낀점 및 배운점은 프로그램의 전체적인 코드만 생각하느라 `#include`나 선언의 정확한 위치에 대해 깊이 고민하지 않았었는데, 헤더파일과 헤더선언에서 오류가 걸리며 제가 어디서 실수를 했고, 그 원인이 무엇인지에 대해 깨달았다는것에 큰 소득이 있는것 같습니다. 특히 상속관계를 직접 코드로 구현 하는 과정에서 자식클래스의 생성자에서 명시적으로 부모클래스를 호출하고, 어떻게 동작되는지 순서의 엄격함을 다시한번 깨닫게 되며 이번 11주차 과제3 소감문을 마치겠습니다.